

PHASE 1

miniHPC

Environment Setup & Networking

Laptop A (Master)

User: GuyThatDoesEveryThin

IP: 192.168.1.10

Laptop B (Worker)

User: Arham

IP: 192.168.1.2

WSL2

Ubuntu

OpenMPI

mpi4py

OpenSSH

■ Table of Contents

1. System Information & Network Setup
2. Phase 1 Overview
3. Step 1 — WSL2 + Ubuntu Installation
4. Step 2 — OpenMPI Installation
5. Step 3 — mpi4py & Python Libraries
6. Step 4 — SSH Setup (Windows OpenSSH)
7. Step 5 — Firewall Configuration
8. Step 6 — Passwordless SSH Key Setup
9. Errors & Solutions Reference
10. Ethernet Alternative Setup

1. System Information & Network Setup

Both nodes are Windows machines connected over Wi-Fi (PTCL Flash Fiber 4G). The cluster uses Wi-Fi IPs for communication in this phase.

	Laptop A (Master)	Laptop B (Worker)
OS	Windows 10	Windows 11
Username	GuyThatDoesEveryThin	Arham
IP Address	192.168.1.10	192.168.1.2
Role	MPI Master / SSH Client	MPI Worker / SSH Server
Subnet Mask	255.255.255.0	255.255.255.0
Gateway	192.168.1.1	192.168.1.1

2. Phase 1 Overview

Phase 1 establishes the complete compute environment on both laptops, enabling distributed MPI jobs to run across the two nodes over the local network. The stack runs inside WSL2 (Ubuntu) on each Windows machine — this gives a real Linux environment where OpenMPI, SSH, and mpi4py behave identically to a production HPC cluster.

#	Step	What it does
1	WSL2 + Ubuntu	Linux runtime on Windows — foundation for everything
2	OpenMPI	MPI library for distributed computation
3	mpi4py + libs	Python MPI bindings + scientific stack
4	OpenSSH (Windows)	Node-to-node SSH communication
5	Firewall rules	Open ICMP (ping) and Port 22 on both nodes
6	SSH Key Auth	Passwordless login between nodes for MPI jobs

3. Step 1 — WSL2 + Ubuntu Installation

Run the following in PowerShell as Administrator on **both laptops**. This installs WSL2 with Ubuntu as the default distro.

```
POWERSHELL (ADMIN)
wsl --install
```

Reboot when prompted. After reboot, Ubuntu will launch and ask you to create a username and password. These become your WSL credentials — keep them safe.

Verify WSL version

```
POWERSHELL
wsl -l -v
```

■ Known Issue: Forgotten WSL Password

✗ **ERROR: Locked out of WSL — forgot Linux user password**
sudo operations fail because the password is unknown.

✓ **FIX: Reset WSL Password via Root**

1. Open PowerShell (Windows)
2. List distros: `wsl -l -v`
3. Enter root: `wsl -d Ubuntu -u root`
4. Reset password: `passwd <yourusername>`
5. Restore default user: `ubuntu config --default-user <yourusername>`

4. Step 2 — OpenMPI Installation

Inside WSL2 Ubuntu terminal on **both laptops**:

```
BASH (WSL2 UBUNTU)
sudo apt update && sudo apt install \
    openmpi-bin openmpi-common openmpi-doc libopenmpi-dev
```

Verify the installation:

```
BASH
mpirun --version
ompi_info | head -5
```

■ Known Issue: Package Not Found

✗ **ERROR: E: Unable to locate package openmpi-dev**
Ubuntu names the dev headers differently — 'openmpi-dev' does not exist.

✓ FIX: Use correct package name

```
sudo apt update && sudo apt install openmpi-bin libopenmpi-dev
If still failing, enable the Universe repo first:
sudo add-apt-repository universe
```

5. Step 3 — mpi4py & Python Libraries

Install the Python scientific stack including mpi4py inside WSL2 Ubuntu on **both laptops**:

```
BASH (WSL2)
pip install mpi4py numpy matplotlib pillow psutil
```

Verify mpi4py works:

```
BASH
python3 -c "from mpi4py import MPI; print('MPI OK, size=', MPI.COMM_WORLD.Get_size())"
```

■ Known Issue: pip not found

✗ ERROR: Command 'pip' not found

Python's package manager isn't included in the default Ubuntu WSL image.

✓ FIX: Install pip

```
sudo apt update && sudo apt install python3-pip
```

■ Known Issue: PEP 668 — Externally Managed Environment

✗ ERROR: error: externally-managed-environment

Ubuntu 24.04 blocks global pip installs to protect the system Python.

✓ FIX: Use --break-system-packages flag

```
pip install mpi4py numpy matplotlib pillow psutil --break-system-packages
Or use a virtual environment (cleaner for a project):
python3 -m venv ~/miniHPC-env
source ~/miniHPC-env/bin/activate
pip install mpi4py numpy matplotlib pillow psutil
```

6. Step 4 — SSH Setup (Windows OpenSSH)

MPI uses SSH to launch processes on remote nodes. We use Windows OpenSSH so that both Windows and WSL2 sessions can SSH between the nodes. Run these steps on **both laptops**.

Quick Diagnosis — Check Current SSH Status

```
POWERSHELL (ADMIN)
# Is SSH Server installed?
Get-WindowsCapability -Online | Where-Object Name -like 'OpenSSH.Server*'

# Is SSH Server running?
Get-Service sshd

# Is anything using port 22?
Get-NetTCPConnection -LocalPort 22 -ErrorAction SilentlyContinue
```

Install & Start OpenSSH Server

```
POWERSHELL (ADMIN)
# Install
Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0

# Start the service
Start-Service sshd

# Auto-start on every boot
Set-Service -Name sshd -StartupType 'Automatic'

# Confirm it is listening on port 22 (should show state: Listen)
Get-NetTCPConnection -LocalPort 22
```

Alternative: Install via Windows Settings

```
WINDOWS
Settings > Optional Features > Add a feature > OpenSSH Server
```

Connect via SSH

```
POWERSHELL / CMD
# From Laptop A, SSH into Laptop B
ssh Arham@192.168.1.2

# From Laptop B, SSH into Laptop A
ssh GuyThatDoesEveryThin@192.168.1.10
```

■ Known Issue: Port 22 Stolen by iphlpsvc / SshpHost

✗ ERROR: SSH connection refused — sshd not owning port 22

Windows services 'iphlpsvc' (IP Helper) or 'SshpHost' often grab port 22 before sshd starts, especially after a reboot.

✓ FIX: Kill the imposter and reclaim port 22

```
# Stop the services stealing port 22
Stop-Service SshpHost -ErrorAction SilentlyContinue
Stop-Service iphlpsvc -ErrorAction SilentlyContinue
# Verify port 22 is now free (should return nothing)
Get-NetTCPConnection -LocalPort 22
# Start the real SSH server
Start-Service sshd
# Confirm sshd owns the port
Get-Process -Id (Get-NetTCPConnection -LocalPort 22).OwningProcess
```

Permanent Fix — Survive Reboots

POWERSHELL (ADMIN)

```
# Disable IP Helper permanently (not needed for IPv4-only clusters)
Set-Service iphlpsvc -StartupType Disabled

# Use delayed-auto so sshd always wins the port race on boot
sc config sshd start= delayed-auto
```

SSH Config File Fix

If SSH keeps rejecting connections, check the config. Open Notepad as Admin and open
C:\ProgramData\ssh\sshd_config

Uncomment these lines:

SSHD_CONFIG

```
Port 22
ListenAddress 0.0.0.0
PubkeyAuthentication yes
PasswordAuthentication yes
```

Comment out these lines (scroll to very bottom):

SSHD_CONFIG

```
#Match Group administrators
#     AuthorizedKeysFile __PROGRAMDATA__/ssh/authorized_keys
```

POWERSHELL (ADMIN)

```
Restart-Service sshd
```

Debugging: Verbose SSH Output

POWERSHELL

```
# Run with -v flag to see exactly where the connection fails
ssh -v Arham@192.168.1.2

# Look for: 'debug1: Expecting SSH2_MSG_KEXINIT'
# If it resets here -> handshake/config issue (go back to sshd_config fix)
```

7. Step 5 — Firewall Configuration

By default, Windows Firewall blocks both ICMP (ping) and SSH (port 22) from other devices. Add inbound rules on **both laptops**. The PowerShell method is preferred.

1. Allow ICMP (Ping)

```
POWERSHELL (ADMIN)
New-NetFirewallRule -DisplayName "Allow ICMPv4 Echo Request" `
  -Direction Inbound -Protocol ICMPv4 -IcmpType 8 -Action Allow
```

2. Allow SSH — Port 22 Inbound

```
POWERSHELL (ADMIN)
New-NetFirewallRule -DisplayName "Allow SSH (Port 22)" `
  -Direction Inbound -Protocol TCP -LocalPort 22 -Action Allow
```

3. Verify Rules Were Created

```
POWERSHELL
Get-NetFirewallRule | Where-Object { $_.DisplayName -match "Allow (ICMP|SSH)" } |
  Select-Object DisplayName, Enabled, Direction, Action
```

Expected output:

```
OUTPUT
DisplayName           Enabled Direction Action
-----
Allow ICMPv4 Echo Request True    Inbound Allow
Allow SSH (Port 22)    True    Inbound Allow
```

■ Known Issue: Public Network Profile Ignores Rules

✗ ERROR: Custom firewall rules ignored — Wi-Fi set to Public

Windows ignores custom inbound rules when the network profile is 'Public'. PTCL Flash Fiber may be detected as a public hotspot.

✓ FIX: Set network profile to Private

```
Get-NetConnectionProfile | Set-NetConnectionProfile -NetworkCategory Private
```

Test Port Reachability Between Nodes

```
POWERSHELL
# From Laptop A — can it reach Laptop B on port 22?
Test-NetConnection -ComputerName 192.168.1.2 -Port 22

# TcpTestSucceeded : True → Port open, but SSH software may be rejecting (fix sshd_config)
# TcpTestSucceeded : False → Firewall is still blocking (re-check rules and network profile)
```


8. Step 6 — Passwordless SSH Key Setup

OpenMPI requires passwordless SSH to launch processes on remote nodes. We generate Ed25519 key pairs and copy public keys between laptops.

Step 1 — Generate Key Pair on Laptop A

```
POWERSHELL (LAPTOP A)

# On Laptop A (GuyThatDoesEveryThin @ 192.168.1.10) - PowerShell
ssh-keygen -t ed25519 -C "miniHPC-LaptopA"

# Prompts:
#   File: press Enter (saves to C:\Users\GuyThatDoesEveryThin\.ssh\id_ed25519)
#   Passphrase: press Enter (no passphrase - required for MPI automation)
```

Step 2 — Copy Public Key to Laptop B

Method A — if ssh-copy-id is available:

```
POWERSHELL (LAPTOP A)

ssh-copy-id Arham@192.168.1.2
```

Method B — Manual (Windows-friendly):

```
POWERSHELL

# On Laptop A: read the public key
Get-Content "$env:USERPROFILE\.ssh\id_ed25519.pub"

# Copy the output (starts with 'ssh-ed25519 ...')

# On Laptop B: paste into authorized_keys
New-Item -ItemType Directory -Path "$env:USERPROFILE\.ssh" -Force
notepad "$env:USERPROFILE\.ssh\authorized_keys"
# Paste the key, save, close Notepad
```

Step 3 — Fix Permissions on Laptop B (Critical!)

Windows SSH is strict about file permissions. Wrong permissions = key ignored silently.

```
POWERSHELL (LAPTOP B — ADMIN)

# Fix .ssh folder permissions
icacls "$env:USERPROFILE\.ssh" /inheritance:r /grant:r "$env:USERNAME:(OI)(CI)F"

# Fix authorized_keys permissions
icacls "$env:USERPROFILE\.ssh\authorized_keys" /inheritance:r /grant:r "$env:USERNAME:F"
```

Step 4 — Test Passwordless Login

```
POWERSHELL (LAPTOP A)

# From Laptop A
ssh Arham@192.168.1.2
# Should connect WITHOUT asking for a password
```

Step 5 — Repeat: Laptop B → Laptop A

```
POWERSHELL (LAPTOP B)

# On Laptop B: generate key
ssh-keygen -t ed25519 -C "miniHPC-LaptopB"

# Copy public key to Laptop A
# (read id_ed25519.pub on Laptop B, paste into Laptop A's authorized_keys)

# Test:
ssh GuyThatDoesEveryThin@192.168.1.10
```

Quick Reference: SSH Commands

From	To	Command
Laptop A	Laptop B	ssh Arham@192.168.1.2
Laptop B	Laptop A	ssh GuyThatDoesEveryThin@192.168.1.10

Troubleshooting: Still Asking for Password?

Open sshd_config on Laptop B and verify:

```
SSHD_CONFIG (C:\PROGRAMDATA\SSH\SSHD_CONFIG)

# These must be uncommented (no # at start):
PubkeyAuthentication yes
PasswordAuthentication yes

# These must be COMMENTED OUT (add # to start):
#Match Group administrators
#    AuthorizedKeysFile __PROGRAMDATA__/ssh/authorized_keys
```

```
POWERSHELL (ADMIN)

Restart-Service sshd
```

9. Errors & Solutions Quick Reference

Error	Cause	Fix
E: Unable to locate package openmpi-bin	Wrong package name on Ubuntu	Use: apt install openmpi-bin libopenmpi-dev
Command 'pip' not found	pip not pre-installed in WSL Ubuntu	apt install python3-pip
error: externally-managed-environment	Ubuntu 24.04 PEP 668 protection	Add --break-system-packages or use venv
SSH Connection Refused	sshd not running or port 22 stolen	Stop iphlpvc/SshpHost, then Start-Service sshd
Firewall rules ignored	Network profile set to Public	Set-NetConnectionProfile -NetworkCategory Private
Still asking for password (SSH key)	Wrong sshd_config or wrong file permissions	Comment out Match Group admins block; fix iclacs permissions
Forgot WSL password	No recovery option in terminal	wsl -d Ubuntu -u root, then passwd <user>
TcpTestSucceeded: False	Firewall blocking port 22	Add New-NetFirewallRule for SSH + set Private profile
debug1: Expecting SSH2_MSG_KEXINIT	SSH handshake / config issue	Restart sshd after fixing sshd_config

10. Ethernet Alternative Setup

■ Ethernet Setup (Alternative to Wi-Fi)

This guide was completed and tested over Wi-Fi. Ethernet is faster, more stable, and avoids Wi-Fi interference — recommended for serious MPI benchmarking.

If connecting the two laptops directly via an Ethernet cable (or through a switch/router), assign static IPs to avoid DHCP conflicts and ensure reliable node addressing.

Assign Static IPs (Both Laptops)

```
WINDOWS NETWORK SETTINGS

# On Laptop A:
# Control Panel > Network & Internet > Network Connections
# Right-click Ethernet adapter > Properties > IPv4 > Use the following IP:
IP Address:    10.0.0.1
Subnet Mask:   255.255.255.0
Gateway:       (leave blank for direct cable, or 10.0.0.1 if using router)

# On Laptop B:
IP Address:    10.0.0.2
Subnet Mask:   255.255.255.0
```

Set Network Profile to Private

```
POWERSHELL (ADMIN)

# Run on both laptops after connecting Ethernet
Get-NetConnectionProfile | Set-NetConnectionProfile -NetworkCategory Private
```

Verify Connectivity

CMD / POWERSHELL

```
# From Laptop A, ping Laptop B  
ping 10.0.0.2
```

```
# From Laptop B, ping Laptop A  
ping 10.0.0.1
```

■ NOTE

After setting Ethernet static IPs, follow the exact same SSH setup steps from Sections 6-8, but replace 192.168.1.10 with 10.0.0.1 (Laptop A) and 192.168.1.2 with 10.0.0.2 (Laptop B).

Updated SSH Commands for Ethernet

POWERSHELL

```
# Laptop A → Laptop B  
ssh Arham@10.0.0.2
```

```
# Laptop B → Laptop A  
ssh GuyThatDoesEveryThin@10.0.0.1
```

■ INFO

Ethernet benefits for miniHPC: Lower latency (sub-1ms vs 5-10ms Wi-Fi), no packet loss from wireless interference, and stable bandwidth for MPI message passing. Use Ethernet for performance benchmarks and final demonstrations.